

Title: Intelligent Chatbot for customer
support using AWS serverless services

By K.YugasaiManikanta - 2200031887



Cloud and Serverless Computing - 22CEC3305A

Section 32

Under the guidance of

Dr. S.Kavitha

Abstract

Abstract

This document presents the architecture and deployment strategy of an AI-powered Telegram bot utilizing a combination of AWS cloud services and open-source AI models. The bot is designed to handle real-time text and image-based interactions, offering intelligent responses through advanced natural language processing and visual understanding capabilities. The solution architecture integrates AWS Lambda, API Gateway, EC2, and DynamoDB, enabling a scalable, serverless backend with efficient compute resources for model inference. Textual intelligence is powered by LLaMA 2, while visual interpretation is handled by Moondream, both running on an EC2 instance with Ollama for lightweight model serving. Telegram acts as the user interface, allowing seamless and intuitive communication. This deployment showcases the synergy of modern cloud infrastructure and open AI technologies in delivering a responsive and intelligent user experience.

Modules

1. Telegram Bot Module

Role: Acts as the primary interface for users to interact with the AI system.

- Receives user messages (text or images).
- Sends responses back after processing.
- Connected via webhook to API Gateway.

2. API Gateway Module (AWS)

Role: Acts as the secure entry point for all incoming requests from Telegram.

- Receives webhook events from Telegram.
- Forwards the request payload to AWS Lambda.
- Handles RESTful API structure and manages routing.

3. AWS Lambda Module

Role: Performs lightweight request processing and orchestration.

- Parses incoming messages and determines message type (text/image).
- Routes requests to appropriate services (e.g., EC2 for inference).
- Sends formatted responses back to Telegram via Bot API.
- Executes without requiring a server, supporting scale on-demand.

4. EC2 AI Inference Module

Role: Hosts and serves AI models using Ollama for both text and image processing.

- LLaMA 2: Processes and generates human-like responses for text queries.
- Moondream: Handles image captioning, object recognition, or visual context interpretation.
- Runs a local API server with Ollama for model inference.
- Provides a powerful compute environment for AI tasks.

● **5. DynamoDB Module**

Role: Manages persistent data storage for user interactions and bot memory.

- Stores session data, chat history, prompt logs, and metadata.
- Enables contextual and stateful conversations.
- Scales automatically with usage.

6. Ollama (on EC2)

Role: Container-based model runner and lightweight serving environment.

- Wraps LLaMA 2 and Moondream models into efficient APIs.
- Simplifies model management and deployment.
- Enhances inference performance and developer productivity.

LAMBDA FUNCTION CODE:

```
curl -X POST
"https://api.telegram.org/bot7688154062:AAHx7_p6pH_XMhgfdz
hAlZKaEM6LQahs5r8/setWebhook" \
  -H "Content-Type: application/json" \
  -d '{"url": "https://91gl1fwyoh.execute-api.ap-south-
1.amazonaws.com/prod/telegram"}'
```

server setup:

```
sudo sudo
sudo yum install -y python3-pip
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &
ollama pull qwen2:1.5b
python3 -m venv ollama-env
source ollama-env/bin/activate
pip3 install fastapi uvicorn "ollama[httpx]"
nano ollama_api.py
```

```
# ollama_api.py
from fastapi import FastAPI, Request
from pydantic import BaseModel
import ollama
```

```
app = FastAPI()
client = ollama.AsyncClient()
```

```
class MessageRequest(BaseModel):
    prompt: str
```

```
@app.post("/generate")
```

```
async def generate_text(req: MessageRequest):
    prompt = req.prompt
    output = ""
    async for chunk in await client.generate(model="qwen2:1.5b",
prompt=prompt, stream=True):
        output += chunk["response"]
    return {"response": output}
```

```
uvicorn ollama_api:app --host 0.0.0.0 --port 8000
```

```
curl -X POST http://107.23.103.66:8000/generate -H "Content-
Type: application/json" -d '{"prompt": "Tell me a joke."}'
```

```
import json
import requests
```

```
TELEGRAM_TOKEN =
"7688154062:AAHx7_p6pH_XMhgfdzhAlZKaEM6LQahs5r8"
TELEGRAM_URL =
f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
```

```
OLLAMA_API_URL = "http://107.23.103.66:8000/generate"
```

```
def lambda_handler(event, context):
    try:
        body = json.loads(event["body"])
        message = body.get("message")

        if not message:
            return {"statusCode": 200, "body": "No message found"}
```

```

chat_id = message["chat"]["id"]
user_text = message.get("text", "")

if not user_text:
    return respond(chat_id, "Please enter a message.")

# Send to FastAPI Ollama endpoint
ollama_response = requests.post(
    OLLAMA_API_URL,
    headers={"Content-Type": "application/json"},
    data=json.dumps({"prompt": user_text}),
    timeout=30
)

if ollama_response.status_code == 200:
    reply_text = ollama_response.json().get("response", "No
response from model.")
else:
    reply_text = f'Ollama error:
{ollama_response.status_code}'

    return respond(chat_id, reply_text)

except Exception as e:
    print("Error:", str(e))
    return {"statusCode": 500, "body": str(e)}

def respond(chat_id, text):
    telegram_payload = {
        "chat_id": chat_id,
        "text": text
    }

```

```
telegram_response = requests.post(TELEGRAM_URL,  
json=telegram_payload)
```

```
return {  
    "statusCode": 200,  
    "body": json.dumps({  
        "message": "Message sent to Telegram",  
        "telegram_status": telegram_response.status_code  
    })  
}
```

```
pip install requests -t .  
Compress-Archive -Path * -  
DestinationPath ../lambda_function.zip
```

```
api gateway:method ->post /telegram stage->prob-> deploy
```

```
curl -X POST  
https://api.telegram.org/bot<YOUR_BOT_TOKEN>/setWebhook  
\  
-d "url=https://abc123xyz.execute-api.us-east-  
1.amazonaws.com/telegram"
```

```
curl -X POST  
https://api.telegram.org/bot7688154062:AAHx7_p6pH_XMhgfdz  
hAlZKaEM6LQahs5r8/setWebhook \  
-d "url=https://8ib9uad4r2.execute-api.us-east-  
1.amazonaws.com/prod"
```

```
curl "https://8ib9uad4r2.execute-api.us-east-  
1.amazonaws.com/?chat_id=5075700464&text=Hello,I am  
yugasai from vijayawada tell me about Vijayawada!+from+GET"
```



```
{  
  "body":  
    "{\\\"message\\\":{\\\"chat\\\":{\\\"id\\\":5075700464},\\\"text\\\":\\\"Hello tell  
me about charminar\\\"}}"  
}
```

ARCHITECTURE DIAGRAM:

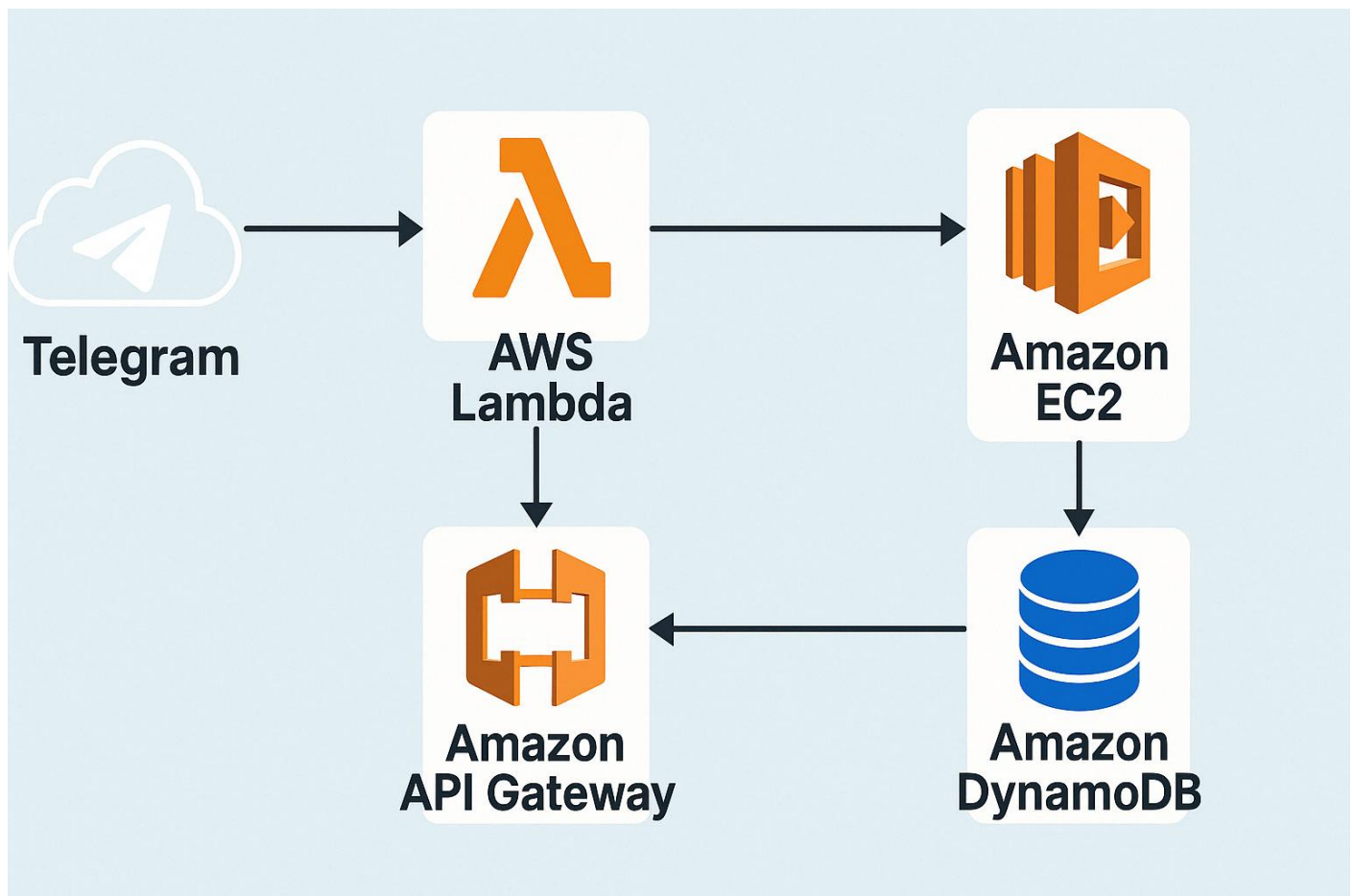


Fig 1: Overview of how AWS services are used in the project

Screenshots:

Fig1:Running middleware logic on lambda that triggers EC2

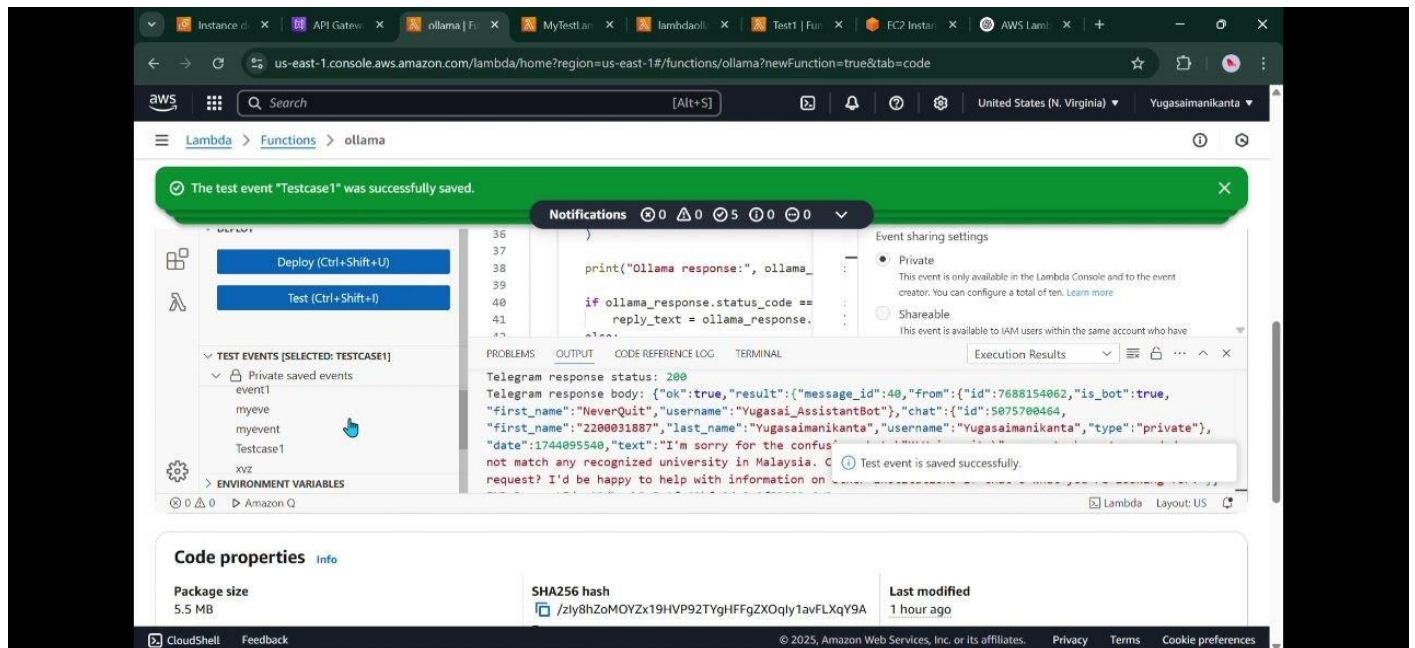


Fig2:

Request Responses in server:

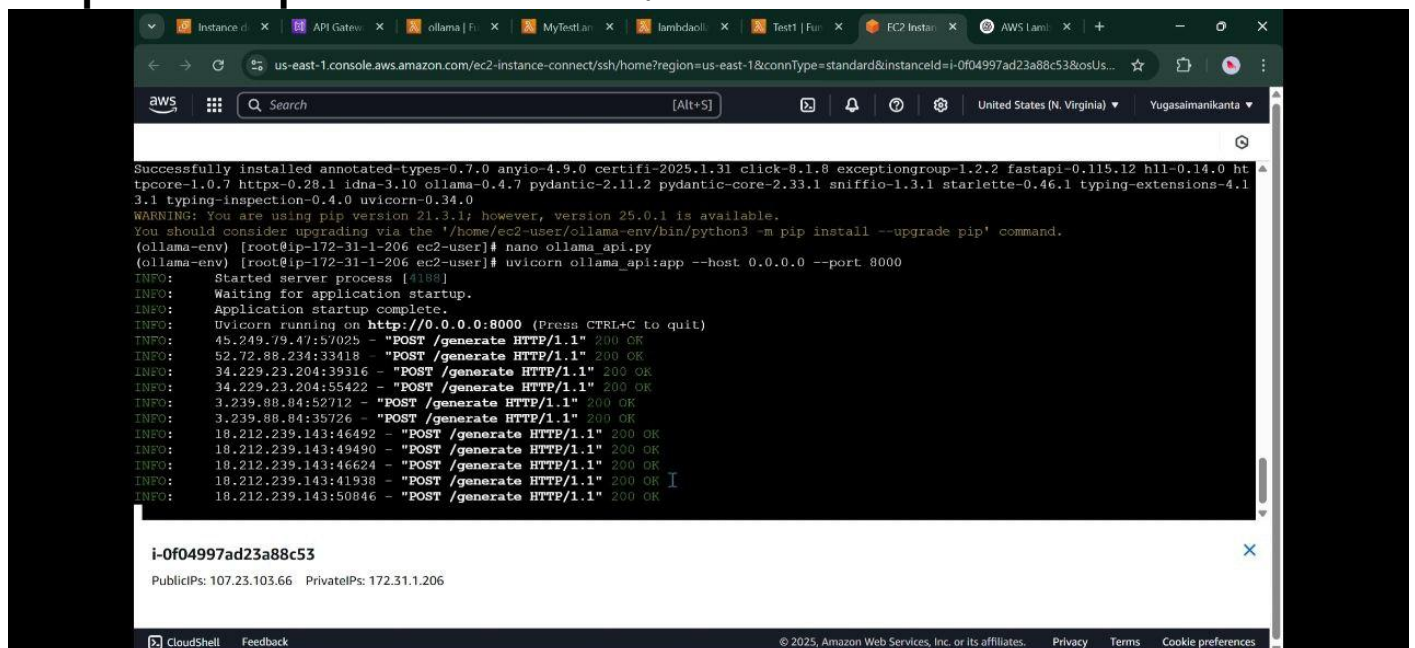


Fig3:Ollama Installation

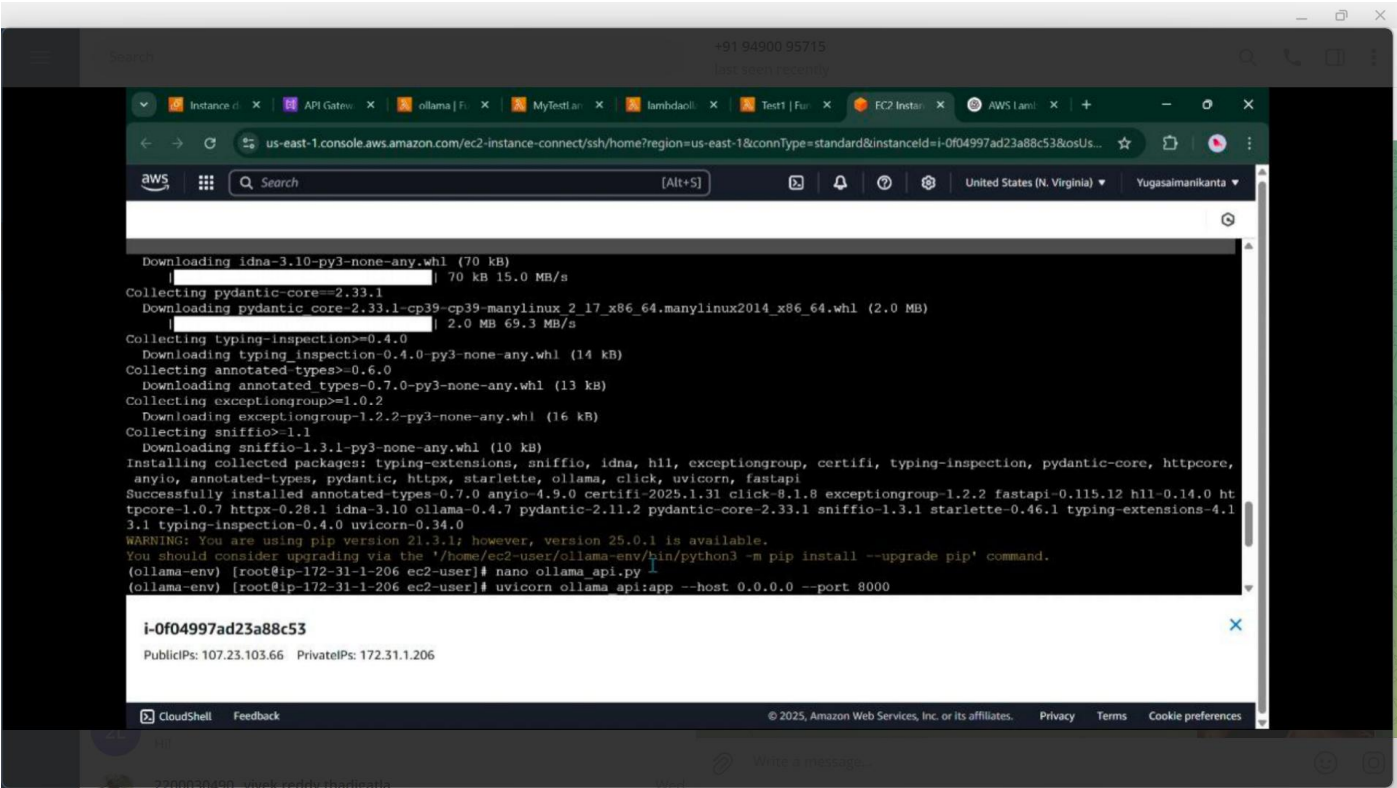
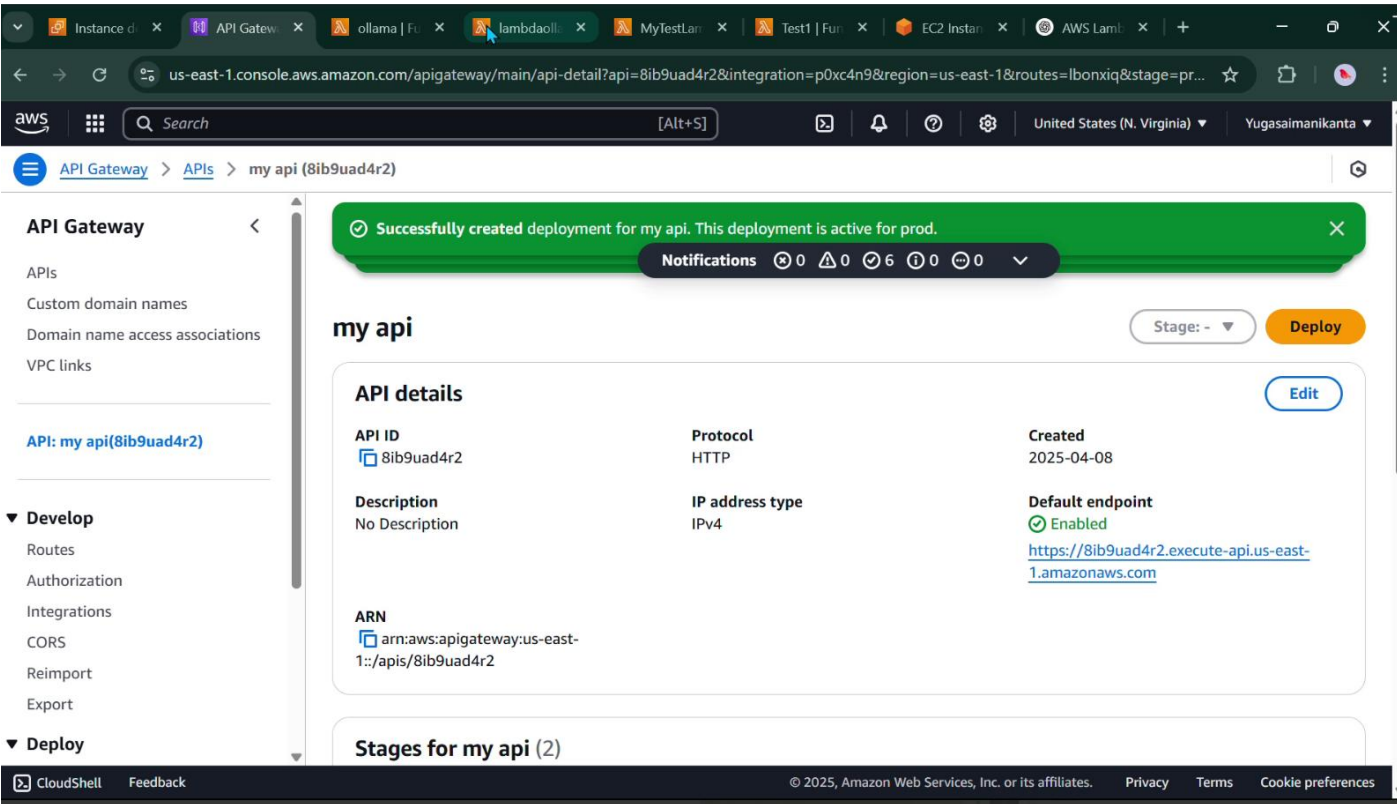


Fig4:API Stages creation for post and get http requests



CONCLUSION

This project demonstrates the seamless integration of cloud-native services and open-source AI models to deliver a powerful and interactive Telegram bot experience. By leveraging the scalability of **AWS Lambda**, the flexibility of **EC2**, the structured access of **API Gateway**, and the persistence of **DynamoDB**, the architecture achieves high performance, reliability, and minimal maintenance overhead. The incorporation of **LLaMA 2** and **Moondream** models via **Ollama** allows the bot to intelligently handle both textual and visual inputs, bringing human-like conversational and perceptual capabilities to a lightweight chat interface.

This deployment not only showcases the potential of AI-driven bots but also sets the stage for future enhancements such as voice input, multilingual support, and extended analytics. The modular and scalable design ensures that the system can grow and adapt to evolving user needs, making it a robust foundation for intelligent, real-time, cross-platform communication.